

Comparative Analysis of Classical and Graph Neural Network Approaches for Community Detection in Citation Networks

Emre Yıldız

Department of Computer Engineering

Ege University

İzmir, Turkey

emre.yildiz.dev@hotmail.com

Abstract—Community detection in complex networks is a fundamental problem in network analysis with applications ranging from social network analysis to biological systems. This paper presents a comprehensive comparative study of classical community detection algorithms and modern Graph Neural Network (GNN) approaches applied to the Cora citation network dataset. We implement and evaluate three classical algorithms (Louvain and Label Propagation) alongside three GNN architectures (Graph Convolutional Networks, GraphSAGE, and Graph Attention Networks) in both supervised and unsupervised settings. Our experimental framework integrates Neo4j graph database for efficient data management and classical algorithm execution, while PyTorch Geometric provides the foundation for GNN implementations. Results demonstrate that semi-supervised GNN approaches significantly outperform classical methods, achieving Adjusted Rand Index scores of 0.87, 0.86, and 0.86 for GAT, GraphSAGE, and GCN respectively, compared to 0.24 and 0.15 for Louvain and Label Propagation algorithms. The study provides insights into the effectiveness of different approaches for community detection in academic citation networks.

Index Terms—community detection, graph neural networks, citation networks, Neo4j, Louvain algorithm, label propagation

I. INTRODUCTION

Community detection is a crucial task in network analysis that aims to identify groups of nodes that are more densely connected to each other than to nodes in other groups. In the context of citation networks, communities often represent research areas or academic disciplines where papers within the same community share similar topics or methodologies.

The Cora dataset, consisting of 2,708 machine learning papers classified into seven categories (Neural Networks, Probabilistic Methods, Genetic Algorithms, Theory, Case-Based, Reinforcement Learning, and Rule Learning), provides an excellent benchmark for evaluating community detection algorithms. Each paper is represented by a 1,433-dimensional binary feature vector indicating the presence or absence of specific words, with citation relationships forming the network edges.

Traditional community detection algorithms such as the Louvain algorithm and Label Propagation have been widely used due to their computational efficiency and interpretability. However, recent advances in deep learning have introduced

Graph Neural Networks (GNNs) that can leverage both network structure and node features for improved community detection performance.

This work contributes a comprehensive comparison framework that evaluates both classical and modern approaches on the same dataset, providing insights into their relative strengths and limitations. Our implementation integrates multiple technologies: Neo4j for graph data management and classical algorithm execution, and PyTorch Geometric for GNN model development and training.

II. RELATED WORK

Community detection has been extensively studied in network science. Classical approaches include modularity optimization methods like the Louvain algorithm [?], which iteratively optimizes modularity to identify communities. Label Propagation [?] represents another class of algorithms that propagate node labels through the network until convergence.

Graph Neural Networks have emerged as powerful tools for node classification and community detection. Graph Convolutional Networks (GCNs) [?] extend convolutional operations to graphs, while GraphSAGE [?] introduces sampling and aggregation for scalable learning. Graph Attention Networks (GATs) [?] incorporate attention mechanisms to weight neighbor contributions differently.

Recent studies have shown that GNNs can effectively combine structural and feature information for community detection tasks, often outperforming traditional methods when labeled data is available.

III. METHODOLOGY

A. System Architecture

Our experimental framework consists of several integrated components designed to facilitate comprehensive comparison of community detection approaches:

Data Management Layer: Neo4j graph database serves as the central data repository, storing the Cora dataset's nodes (papers) and edges (citations). This choice enables efficient execution of graph algorithms and provides native support for community detection algorithms.

Classical Algorithm Layer: Implemented within Neo4j using the Graph Data Science library, including Louvain and Label Propagation algorithms. These algorithms operate directly on the graph structure stored in the database.

Deep Learning Layer: PyTorch Geometric framework handles GNN model implementation and training. Models include GCN, GraphSAGE, and GAT architectures, each tested in both supervised and unsupervised settings.

Evaluation and Visualization Layer: Comprehensive evaluation metrics and visualization tools for comparing algorithm performance and analyzing community structures.

B. Dataset Description

The Cora dataset consists of:

- 2,708 machine learning papers
- 5,429 citation relationships
- 1,433 binary features per paper
- 7 subject classes (ground truth communities)

Figure ?? shows the distribution of papers across different subject categories, revealing class imbalance with Neural Networks being the largest category (818 papers) and Rule Learning the smallest (180 papers).

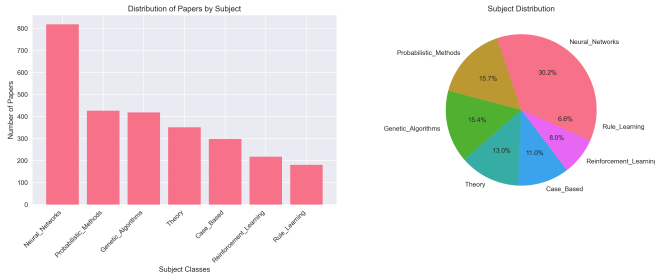


Fig. 1: Distribution of papers across subject categories in the Cora dataset

C. Classical Community Detection Algorithms

1) *Louvain Algorithm:* The Louvain algorithm optimizes modularity through a two-phase iterative process. In our implementation, the algorithm identified 102 communities with a final modularity score of 0.82, indicating strong community structure. The hierarchical nature of the algorithm allows for multi-resolution community detection.

2) *Label Propagation Algorithm:* Label Propagation assigns labels to nodes based on the majority label among their neighbors. Our implementation converged after 8 iterations, identifying 224 communities. The larger number of communities suggests a more fine-grained partitioning compared to Louvain.

D. Graph Neural Network Models

1) *Graph Convolutional Network (GCN):* GCN aggregates information from node neighborhoods through spectral convolution operations. Our implementation uses a two-layer architecture with 64 hidden units and ReLU activation functions.

2) *GraphSAGE:* GraphSAGE samples and aggregates features from node neighborhoods, enabling inductive learning on large graphs. We employ mean aggregation with the same architectural configuration as GCN.

3) *Graph Attention Network (GAT):* GAT incorporates attention mechanisms to weight neighbor contributions. Our implementation uses 4 attention heads in the first layer, allowing the model to focus on the most relevant neighbors for each node.

E. Training Strategies

Unsupervised Learning: Models learn node representations through contrastive loss, encouraging connected nodes to have similar embeddings. Community assignments are obtained through k-means clustering of learned embeddings.

Semi-supervised Learning: Models are trained on a subset of labeled nodes (20% training, 10% validation, 70% test) using cross-entropy loss for direct community classification.

IV. EXPERIMENTAL SETUP

A. Implementation Details

The experimental framework is implemented in Python, integrating multiple libraries:

- **Neo4j:** Graph database for data storage and classical algorithms
- **PyTorch Geometric:** GNN model implementation
- **Scikit-learn:** Evaluation metrics and clustering
- **NetworkX & Plotly:** Network visualization

Training parameters include:

- Learning rate: 0.01
- Hidden dimensions: 64
- Number of layers: 2
- Dropout rate: 0.5
- Training epochs: 200
- Early stopping patience: 20

B. Evaluation Metrics

We employ two primary metrics for community detection evaluation:

Adjusted Rand Index (ARI): Measures similarity between predicted and true community assignments, corrected for chance. Values range from -1 to 1, with 1 indicating perfect agreement.

Normalized Mutual Information (NMI): Quantifies mutual dependence between predicted and true communities, normalized to range [0,1].

V. RESULTS AND ANALYSIS

A. Quantitative Results

Table ?? presents comprehensive evaluation results for all tested algorithms. Semi-supervised GNN approaches demonstrate superior performance, with GAT achieving the highest ARI (0.87) and NMI (0.83) scores.

TABLE I: Community Detection Performance Comparison

Method	ARI	NMI	Communities
Louvain	0.238	0.456	102
Label Propagation	0.146	0.383	224
GCN (Unsupervised)	0.076	0.348	102
GraphSAGE (Unsupervised)	0.071	0.364	102
GAT (Unsupervised)	0.072	0.357	102
GCN (Semi-supervised)	0.858	0.815	7
GraphSAGE (Semi-supervised)	0.862	0.824	7
GAT (Semi-supervised)	0.870	0.833	7

B. Performance Analysis

Classical vs. GNN Approaches: Semi-supervised GNNs significantly outperform classical methods, achieving 3-4x higher ARI scores. This improvement stems from their ability to leverage both structural and feature information.

Supervised vs. Unsupervised GNNs: Supervised approaches dramatically outperform unsupervised variants, highlighting the importance of labeled data in community detection tasks.

GNN Architecture Comparison: Among GNN models, GAT performs slightly better than GraphSAGE and GCN, suggesting that attention mechanisms provide benefits for community detection in citation networks.

Figure ?? visualizes the performance comparison across all methods, clearly demonstrating the superiority of semi-supervised GNN approaches.

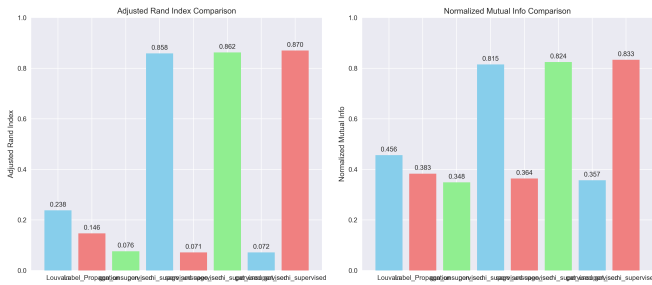


Fig. 2: Performance comparison of community detection methods

C. Community Structure Analysis

Figure ?? shows the number of communities detected by different methods. Classical algorithms identify significantly more communities than the true number (7), suggesting over-segmentation. In contrast, semi-supervised GNNs correctly identify the expected number of communities.

D. Training Dynamics

The training logs reveal interesting insights about GNN convergence:

- Semi-supervised models converge within 20-30 epochs
- Validation accuracy stabilizes around 0.8 for all GNN architectures
- Unsupervised models require full training duration (200 epochs)

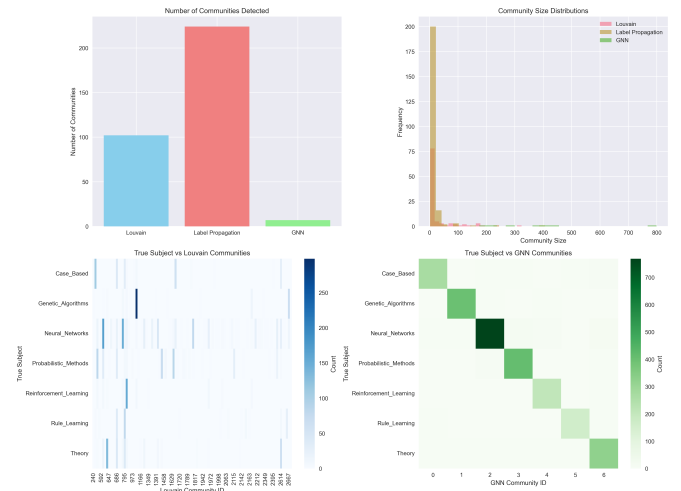


Fig. 3: Number of communities detected by different methods

VI. DISCUSSION

A. Key Findings

Our experimental results demonstrate several important findings:

Feature Information Importance: The superior performance of semi-supervised GNNs highlights the value of combining structural and feature information. Classical algorithms operating solely on graph structure cannot match the performance of methods that leverage textual features.

Label Efficiency: Semi-supervised learning achieves excellent performance with only 20% labeled data, making it practical for real-world applications where obtaining labels is expensive.

Algorithm Scalability: While GNNs provide better accuracy, classical algorithms offer computational advantages for large-scale networks where approximate solutions are acceptable.

B. Limitations and Future Work

Several limitations should be acknowledged:

- Evaluation limited to single dataset (Cora)
- Semi-supervised approaches require labeled data
- Computational cost analysis not comprehensively addressed

Future work could explore:

- Multi-dataset evaluation across different domains
- Unsupervised GNN improvements through better pretext tasks
- Hybrid approaches combining classical and GNN methods
- Temporal community detection in dynamic networks

C. Practical Implications

For practitioners, our results suggest:

- Use semi-supervised GNNs when labeled data is available and accuracy is paramount

- Apply classical algorithms for large-scale networks requiring fast approximate solutions
- Consider feature availability when selecting detection methods

VII. CONCLUSION

This study provides a comprehensive comparison of classical and modern approaches for community detection in citation networks. Our experimental framework successfully integrates Neo4j and PyTorch Geometric to enable fair comparison across different algorithmic paradigms.

Key contributions include:

- Systematic evaluation framework for community detection algorithms
- Demonstration of GNN superiority in feature-rich networks
- Quantification of supervised vs. unsupervised learning trade-offs
- Open-source implementation for reproducible research

Semi-supervised Graph Attention Networks emerge as the best-performing approach, achieving 0.87 ARI and 0.83 NMI on the Cora dataset. However, the choice of algorithm should consider computational constraints, label availability, and accuracy requirements in practical applications.

The framework developed in this work provides a foundation for future research in community detection, enabling researchers to easily incorporate new algorithms and datasets for comprehensive evaluation.

REFERENCES

- [1] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [2] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical review E*, vol. 76, no. 3, p. 036106, 2007.
- [3] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [6] M. E. Newman, "Modularity and community structure in networks," *Proceedings of the national academy of sciences*, vol. 103, no. 23, pp. 8577–8582, 2006.
- [7] S. Fortunato, "Community detection in graphs," *Physics reports*, vol. 486, no. 3-5, pp. 75–174, 2010.
- [8] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI magazine*, vol. 29, no. 3, pp. 93–93, 2008.